

Chapter 8 - The ULA Display

The ULA reproduces the picture model of the ZX Spectrum: a single 256×192 monochrome bitmap painted in pairs of colours chosen per 8×8 cell from a small palette. It is the simplest of the six picture sources, and the easiest to drive directly from BASIC. The ULA sits on compositor layer 15, so it draws on top of every other source except Voodoo 3D.

8.1 What the ULA can show

Item	Value
Display area	256×192 pixels
Character grid	32×24 cells of 8×8
Border	32 pixels on every side
Total frame	320×256 pixels
Colours	15 unique (8 base + 8 bright, but black has no bright form)
VRAM	6912 bytes (6144 bitmap + 768 attributes)
Flash rate	Toggle every 32 frames (about 1.6 Hz at 50 Hz)

Every pixel is either INK (foreground) or PAPER (background). The choice of INK and PAPER comes from an **attribute byte** that covers a whole 8×8 cell, so any two adjacent cells can use different colour pairs but pixels within one cell are limited to two colours. This is the Spectrum's famous **attribute clash**, and the ULA reproduces it exactly.

8.2 BASIC keywords

ULA introduces every ULA subcommand.

Form	Effect
ULA ON / ULA OFF	Enable / disable the chip.
ULA BORDERcolour	Write <i>colour</i> (0–7) to the border register.
ULA INKcolour	Set the current foreground colour (0–7).
ULA PAPERcolour	Set the current background colour (0–7).
ULA BRIGHTflag	Set the current bright flag (0 or 1).
ULA FLASHflag	Set the current flash flag (0 or 1).
ULA PLOT x, y	Set the bitmap pixel at (x, y) using the ZX-style address.
ULA CLS [<i>attr</i>]	Clear the bitmap to zero and fill the attribute area with <i>attr</i> (default 0×38 = white paper, black ink).
ULA ATTR $x, y, attr$	Write the attribute byte at character cell (x, y).

A minimal BASIC program that draws a red diagonal on a black background. The attribute byte chosen for ULA CLS decides the colour every PLOT will appear in, because PLOT only sets bits in the bitmap - the colour comes from the cell's attribute:

```

10 ULA ON
20 ULA CLS &H02           : REM black paper (bits 5-3 = 0), red ink (bits 2-0 = 2)
30 FOR I=0 TO 191
40 ULA PLOT I, I
50 NEXT I

```

To change colours under an existing picture, write new attribute bytes with ULA ATTR after plotting.

ULA INK, ULA PAPER, ULA BRIGHT, and ULA FLASH set BASIC's working attribute state but do **not** touch the attribute area themselves. The state is consumed by routines that write characters or whole rectangles. To set the colour of a specific cell directly, use ULA ATTR.

8.3 The register block

The ULA's register block is small. It runs 0xF2000–0xF2017, six 32-bit words. Only the low byte of each register is meaningful.

Address	Name	Purpose
0xF2000	ULA_BORDER	Border colour. Bits 0–2; upper bits ignored.
0xF2004	ULA_CTRL	Master enable and IRQ enable.
0xF2008	ULA_STATUS	Status bits (read-only).
0xF200C	ULA_ADDR_LO	Low byte of paged VRAM address latch.
0xF2010	ULA_ADDR_HI	Upper bits of the 13-bit VRAM address latch.
0xF2014	ULA_DATA	Read/write the VRAM byte at the latched address.

8.3.1 ULA_CTRL bits

Bit	Name	Meaning
0	ENABLE	Master enable.
1	VBANK_IRQ_EN	Raise an interrupt when vertical blank begins.
2	AUTO_INC	Auto-increment the ULA_DATA latch after each access.

8.3.2 ULA_STATUS bits

Bit	Name	Meaning
0	VBANK	Set during the vertical-blank interval.

8.3.3 The paged VRAM port

The ULA exposes its VRAM in two ways. The full 6912-byte VRAM window is mapped directly at 0xFA000–0xFBAFF (see §8.4), but the chip also offers a paged port for CPUs that cannot reach that window cheaply:

1. Write the byte offset into the latch with ULA_ADDR_LO and ULA_ADDR_HI (13-bit value, 0–6911).
2. Read or write ULA_DATA to access that byte.
3. If AUTO_INC is set in ULA_CTRL, the latch advances after every access.

This is the only path on chips with a small address space (for example the 6502 and Z80, which use the equivalent port-I/O mappings).

8.4 The VRAM aperture

The full 6912 bytes of VRAM are mapped at 0xFA000–0xFBFAFF for 32-bit and 64-bit CPUs. The layout is the Spectrum standard:

Range	Size	Contents
0xFA000–0xFB7FF	6144 B	Bitmap.
0xFB800–0xFBFAFF	768 B	Attribute bytes.

8.4.1 Bitmap addressing

The bitmap is not stored in a simple row-major layout. For pixel (x, y) the byte address inside the bitmap is:

```
addr = ((y & 0xC0) << 5)      ; row group  (0, 64, 128)
      | ((y & 0x07) << 8)      ; pixel row inside row group
      | ((y & 0x38) << 2)      ; character row inside row group
      | (x >> 3)               ; column
```

The bit inside the byte is 7 - (x & 7) (bit 7 is the leftmost pixel of the eight). This is the same arrangement as the original Spectrum, and it is the reason a ULA PLOT keyword exists at all: the address arithmetic above is awkward to write each time by hand.

8.4.2 Attribute bytes

Each attribute byte covers one 8 × 8 cell, in row-major order:

```
bit 7 6 5 4 3 2 1 0
    F B P P P I I I
```

Bits	Field	Meaning
0–2	INK	Foreground colour, 0–7.
3–5	PAPER	Background colour, 0–7.
6	BRIGHT	Intensify both INK and PAPER.
7	FLASH	Swap INK and PAPER every 32 frames.

The eight base colours are:

0 Black	4 Green
1 Blue	5 Cyan
2 Red	6 Yellow
3 Magenta	7 White

BRIGHT brightens every colour except black, which is why the total comes to 15 unique colours and not 16.

8.5 Interrupts

The vertical-blank interval is the cleanest time to update VRAM, because the picture is not being scanned. Two ways to detect it:

- Poll ULA_STATUS bit 0.
- Enable VBLANK_IRQ_EN in ULA_CTRL; the CPU then receives an interrupt at the start of each VBlank.

The Spectrum-style INT vector for VBlank on the x86 coprocessor is 0x20; other CPUs receive the interrupt through their normal controllers.

8.6 Putting it together

The shortest path to a Spectrum-style picture from machine language:

1. Write 1 to ULA_CTRL to enable the chip.
2. Clear the bitmap and write the attributes you want.
3. Plot pixels by writing into the bitmap with the formula above, or by setting one bit at a time through ULA_DATA.
4. Change the border colour with ULA_BORDER for cheap visual effects (loading-screen colours, music-driven flashes).

The next chapter covers the Voodoo 3D rasterizer, the top of the compositor stack on layer 20.